

Appendix B

The Multi-Agent Engineering Toolkit

This book hands you a working tool at almost every turn: a test for whether a problem even wants multiple agents, a contract that makes an agent composable, a decision path for picking a topology, a register that names who owns each production risk. The tools live where they were built, spread across the book, so a reader mid-design has to remember which chapter held the right one. This appendix gathers them in one place.

The tools are grouped by the job you reach for them to do, which you know before you know the chapter. The groups follow the work from the first design decision to the roadmap:

- **Deciding what to build** picks the right approach: whether to build a multi-agent system (MAS) at all, and then the choice at each layer of the design, from the framework and reasoning strategy to the topology, protocol, and deployment mechanism.
- **Specifying agent contracts** pins down each boundary in writing: what an agent may decide, what a tool may do, what a message or a task carries across, and what one agent or session hands the next.
- **Verifying production readiness** gates a system before it ships and on each change after, confirming nothing critical is missing.
- **Diagnosing and recovering from failure** finds the cause when something breaks, the pattern that prevents it, and the procedure that recovers from it.
- **Securing and governing change** defends the system against attack, governs what it keeps and learns from, and controls how it is allowed to evolve.
- **Planning the maturity roadmap** decides how far to invest and what to build next.

Within each group the tools run in chapter order. Read in sequence, the groups answer the question an engineer brings to the index: *I am designing a MAS; which tool do I reach for first, second, and third?*

Deciding what to build

The *Chapter 1* test comes first in Table B.1: it settles whether the problem wants several agents at all. The rows after it each settle one choice the design needs, from what a framework decides for you to whether a heavier deployment mechanism is warranted yet, and the last reads a whole system, deployed or proposed, through five questions.

Artifact	Chapter	Use when	Inputs	Output	Failure it prevents
Four-criterion MAS test	<i>Chapter 1, When should you use multi-agent systems?</i>	Deciding whether a problem needs a MAS at all, or fewer agent boundaries will do	Specialization, parallelism, independent verification, runtime coordination	A go/no-go on MAS, and which criteria carried it, under the principle of minimal viable structure	Architectural overreach: agent boundaries that pay coordination cost and buy no system property

Artifact	Chapter	Use when	Inputs	Output	Failure it prevents
Framework commitments checklist	<i>Chapter 3, Table 3.1 and The abstraction stack; Chapter 10, What a good comparison looks like, for the six comparison dimensions and exit-cost overlay, and Table 10.6 for each framework's vocabulary on that stack</i>	Evaluating an unfamiliar framework	How it handles nine elements, from agents to deployment surface	For each element, what the framework decides and what production risk still lands on you; the most opinionated rows are where leaving costs most	Adopting a framework without seeing the operational decisions it bakes in

Artifact	Chapter	Use when	Inputs	Output	Failure it prevents
Portability checklist	<i>Chapter 3, Portability checklist</i>	Committing to a framework, and again when a migration is first raised	The agent step contracts, what the eval suite asserts on, the trace format and its exporter, the adapter layer between business logic and the framework's runner	A pass or fail on each of four checks, and whether migration is a bounded, testable rewrite	Migration cost that scales with the size of the codebase
Reasoning-strategy selection guide	<i>Chapter 4, Table 4.2</i>	Choosing a reasoning strategy for an agent	Task uncertainty, tool dependence, verification availability, cost	A strategy: Direct, Chain-of-Thought, ReAct, Reflexion, or Tree of Thoughts	Oversized reasoning machinery burning tokens for no gain

Artifact	Chapter	Use when	Inputs	Output	Failure it prevents
Message-format selection guide	Chapter 5, Table 5.1	Choosing how much structure a message between agents needs	Boundary risk: asynchronous delivery, independent deployment, changing participants, irreversible side effects, organizational separation	A format: unstructured, structured, or hybrid (a typed envelope around a natural-language payload, the production default)	Plausible wrong answers from prompts nothing validates, or a rigid schema that taxes the reasoning it wraps
Pattern selection framework	Chapter 6, Tables 6.1–6.2 and the seven-question walk under <i>A decision framework for pattern selection</i>	Choosing a collaboration pattern for a problem	Task decomposability, sub-task independence, verification availability, latency budget, cost budget, conflict risk, auditability requirement	A shortlist of candidate patterns, compared across five quality dimensions, then one pattern or a composition	A pattern mismatched to the problem's structure

Artifact	Chapter	Use when	Inputs	Output	Failure it prevents
Aggregation strategies	Chapter 6, <i>Aggregation strategies</i> ; Chapter 8, <i>Result aggregation</i> , for the aggregator signature and two more strategies	Choosing how a fan-out's several outputs become one answer	The task type: creative, factual, or one where quality varies widely across specialists	A strategy: voting, synthesis, or ranked selection; at the orchestration layer, also weighted combination or union with deduplication	Specialists that share assumptions voting unanimously for a wrong answer, or a strong answer diluted by averaging it with weak ones

Artifact	Chapter	Use when	Inputs	Output	Failure it prevents
Pattern composition grammar	Chapter 6, A composition grammar	Combining two or more collaboration patterns into one pipeline	The patterns composed, whether each join chains or nests, and the per-task call budget	Four rules applied: calls counted before committing , one verification layer after generation, one termination owner wired to a hard budget ceiling, and separately scoped local and global verification	Runaway cost, ambiguous termination , or verification gaps

Artifact	Chapter	Use when	Inputs	Output	Failure it prevents
Read-many, write-one rule	<i>Chapter 7, What crosses the agent boundary</i>	Deciding what each spawned sub-agent's context starts with and what comes back	Whether the sub-agents read or write	Context isolation with compressed summaries for read-heavy fan-out, and a single context that sees everything for the write	Sub-agents working from briefs whose conflicting implicit decisions surface only when their outputs meet
Short-term memory compaction trade-offs	<i>Chapter 7, Table 7.3</i>	Choosing how to handle conversational context overflow	The agent's work pattern, the information loss it can tolerate, latency budget, token budget	A strategy: truncate, summarize, selective prune, platform-managed, or trained	Context overflow, or a strategy that drops detail the task needed or pays for a summarization call the latency budget cannot afford

Artifact	Chapter	Use when	Inputs	Output	Failure it prevents
Orchestration topology decision path	<i>Chapter 8, the six-question walk under Choosing your topology, landing on Table 8.2</i>	Selecting a coordination topology	Workflow predictability, consistency needs, failure resilience, scale, whether the agent roster is fixed or open, planning-latency budget	One of five positions from fully centralized to fully decentralized, or a hybrid	A topology mismatch that bottlenecks throughput or blocks recovery
Protocol selection guide	<i>Chapter 9, Table 9.3</i>	Choosing a cross-boundary agent protocol	Tool reuse across boundaries, delegation and task-lifecycle needs, task duration, organizational boundaries, single-team ownership	A protocol: Model Context Protocol (MCP), Agent-to-Agent (A2A), or Agent Network Protocol; or minimal HTTP or a direct call while one team owns every agent	A protocol mismatched to the requirement

Artifact	Chapter	Use when	Inputs	Output	Failure it prevents
Framework selection decision guide	<i>Chapter 10, the two-stage sequence in the chapter opening, Table 10.7, the Scenario-based selection cards, the five steps under Framework selection decision guide, and Common pitfalls in framework evaluation</i>	Choosing a framework under project constraints	Workflow shape, cloud platform, model commitment or open-source models, and checkpoint, guardrail, persistent-memory, streaming, security, and multimodal requirements	A shortlist per constraint; for six common scenarios, the tempting wrong choice and the migration risk; and the structural signals that re-open the choice	Framework regret found mid-project after code is locked into its idioms
Lighter deployment substitutes	<i>Chapter 15, Table 15.2</i>	Deciding whether a heavier deployment mechanism is warranted yet	The mechanism in question, one of seven from a leased queue to a four-stage pipeline with canary	A lighter substitute and the condition under which it stops being enough	A heavier mechanism built before it is warranted, or a substitute run past the point where it holds

Artifact	Chapter	Use when	Inputs	Output	Failure it prevents
Five case-study questions	<i>Chapter 16</i> , the chapter opening, with its evidence rules; Table 16.3 (worked across four cases); run forward in <i>Case study 8 of Appendix D</i>	Reading a deployed system for what transfers, or deriving an architecture for your own workflow	The system or workflow, and its baseline, the simpler system it has to beat	Five answers: where authority sits, what evidence crosses each boundary, how effects are contained, which bottleneck binds, and what decision an evaluation controls	A design that assumes each responsibility needs an agent, or a measure that only fills a dashboard

Table B.1: The tools for deciding what to build

Specifying agent contracts

The book calls a lot of things contracts, and the word means something slightly different each time. Six of them are one family all the same, and every member does the same job at a different boundary: it names the boundary and states what may cross it, so the boundary is explicit and testable rather than assumed. The autonomy contract binds an agent to a decision scope; the task contract, to one job; the interface contract, to a callable shape; the handoff contract, to a delegation between two agents; the agent contract, to callers outside its runtime; and the Evolution Contract binds the system to the changes it may make to itself. *Chapter 4* sets all six side by side under *The agent contract*, the section that defines the interface contract. Five of them are rows in Table B.2, and the Evolution Contract sits with the controls under *Securing and governing change* below, because governing change is what a control does.

The table's other five rows are not contracts. They specify the rest of what a boundary needs: the principles a tool's design follows, the envelope every message carries, the lifecycle a

delegated task reports, the record one session leaves the next, and the envelope that takes work across a protocol boundary. Some of these names sit close together. The task contract says what one job requires of an agent; the task envelope is the format a work request takes on its way to another agent. The handoff contract governs a delegation between two running agents, and the session handoff record is what a finished session leaves a successor that cannot ask it anything. *Chapter 9's* layer-boundary glossary (Table 9.5) separates the pairs that remain: message envelope from task envelope, handoff contract from task envelope, and agent contract from autonomy contract.

Artifact	Chapter	Use when	Inputs	Output	Failure it prevents
Bounded autonomy contract	<i>Chapter 2, The five-component autonomy contract and Figure 2.1 (worked example), with Human-in-the-loop escalation as a design primitive for the escalation pathway</i>	Bounding what a single agent may decide	Role, goal, observable state, decision scope, feedback loop	A written five-component contract per agent, with its escalation trigger and recipient; the human-in-the-loop escalation pathway adds the approval workflow, its timeout, and logged, sampled review	Same-role agents taking different paths to the same goal, or every decision queuing behind a central coordinator

Artifact	Chapter	Use when	Inputs	Output	Failure it prevents
Tool-design principles	<i>Chapter 4, Designing effective tools: capability governance</i>	Exposing a capability to an agent as a tool	The tool's scope, description, input and output schemas, and implementation	A tool that meets five principles: single responsibility, clear description, typed schemas, graceful error handling, and safety boundaries the tool enforces itself	The model's text becoming an unaudited path to system-level side effects
Task contract	<i>Chapter 4, Stage 0: the task contract drives component choices and Table 4.3 (worked example), tested in Stage 3: a minimal test harness</i>	Specifying one agent's job before writing any of it	What the agent must accomplish, what it is allowed to do, and what counts as failure	A five-field contract: output, acceptable uncertainty, permitted sources, failure condition, escalation trigger	Component choices made before anyone agreed what the agent owes

Artifact	Chapter	Use when	Inputs	Output	Failure it prevents
Agent interface contract	<i>Chapter 4, Abstraction thinking: inputs, outputs, and behaviors; Chapter 2, Designing modular agent interfaces, for an interface's five properties</i>	Making an agent composable and swappable	The agent's inputs, outputs, and behavioral guarantees	A framework-agnostic boundary you can test against	An agent that cannot be tested alone, swapped for another, or carried to another framework
Handoff contract	<i>Chapter 5, The handoff contract, worked through under The repaired exchange (Table 5.2) and enforced by Contract tests</i>	Designing one agent's delegation to another	The work being delegated, and the agent receiving it	A complete handoff: claim, provenance, uncertainty, requested action, authority boundary, deadline, expected response format	Ambiguous delegation and lost context: a receiver guessing at whatever the sender left out

Artifact	Chapter	Use when	Inputs	Output	Failure it prevents
Message envelope	<i>Chapter 5, The envelope: identity and lineage</i>	Deciding what every message between agents carries so it can be routed, deduplicated, expired, and traced	Sender, receiver, message type, payload schema version, and deadline	A frozen envelope around the payload: message, conversation, and causation IDs, creation time, absolute expiry, and W3C trace context, validated at the boundary	No way to match a result to its request, tell a duplicate from a retry, or reconstruct a failure
Communicated-task lifecycle	<i>Chapter 5, Figure 5.2</i>	Agreeing on the states a delegated task passes through and the endings a sender must tell apart	A delegated task, and how it ended	Created, accepted, and working, and five terminal states: succeeded, rejected, failed, canceled, and expired	Every unhappy ending reported as "error", leaving the sender one bucket and no policy

Artifact	Chapter	Use when	Inputs	Output	Failure it prevents
Session handoff record	<i>Chapter 8, The session handoff</i>	Work outlasts one context window, and a successor session that remembers nothing must resume it	The goal, and a definition of done per work unit written before any session starts	A record on disk of three things: a specification of what done means and a progress ledger in which no session verifies its own unit (together the two-ledger pattern, persisted), and the resume command	A run that declares itself finished on its own say-so, or sessions lost to rediscovering how to start the environment
Agent contract	<i>Chapter 9, The agent as a service contract</i>	Publishing an agent to callers outside its own runtime	The agent's identity, capabilities, interface shape, constraints, and discovery metadata	A discovery-ready description a registry can advertise and a stranger can call	Framework-coupled integrations that break when the peer is rewritten

Artifact	Chapter	Use when	Inputs	Output	Failure it prevents
Task envelope	<i>Chapter 9, The unified task envelope, with The adapter interface that translates it</i>	Sending work to an agent across a protocol boundary	The instruction, input data, and context, the latency, cost, and priority limits, and whether repeating the step is safe	One protocol-agnostic work request carrying its constraints, its own cross-protocol lifecycle state (submitted, working, completed, failed, or canceled), lineage, and idempotency key, which adapters translate to each wire format	An orchestrator turned translation layer across task formats, or an unprotected side effect retried across a boundary

Table B.2: The tools for specifying agent contracts

Verifying production readiness

Table B.3 lists the gates to walk before the system carries real traffic, and again when it changes. The checklists, most of them the closing design checklist of the chapter that built the capability, each turn "it seems to work" into a list of things that are demonstrably true. The other rows build what those gates rest on: the evaluation specification, suite, and judge that produce the evidence a release is judged on, and the plans, tests, and requirement lists a deployment has to meet.

Artifact	Chapter	Use when	Inputs	Output	Failure it prevents
MAS design-review checklist	<i>Chapter 2, The minimum viable design review (Table 2.3) as the first pass, then the Diagnostic checklist, with each principle's Best practices block for its design commitment, signals, and first diagnostic move, and Principles in tension for the trade-offs</i>	Before scheduling a design review, and at the architecture review before the system ships	A proposed design and its requirements	A first pass that returns ready, not yet, or stop, then gaps named against each of the seven principles	A design that ships with a principle nobody examined

Artifact	Chapter	Use when	Inputs	Output	Failure it prevents
Agent construction checklist	Chapter 4, <i>The agent construction checklist</i>	Before deploying any agent, and again whenever its role, tools, or deployment context changes	The built agent and its system prompt	A pass over 11 items: role, goal, and reasoning strategy; tools and their safety boundaries; memory; error handling, budget, and termination; untrusted tool output; and the single-agent telemetry floor	Deploying an agent with no error handling or observability

Artifact	Chapter	Use when	Inputs	Output	Failure it prevents
Communication design checklist	Chapter 5, <i>The communication design checklist</i>	Before deploying a multi-agent communication layer	The message contracts, routing, and delivery semantics	A pass over 17 items in four groups: meaning; identity; authority and timing; and reliability	Intent the receiver cannot recover, a result that cannot be matched to its request, authority taken from the message instead of server-side grants, a late result discarded silently, or a duplicate effect

Artifact	Chapter	Use when	Inputs	Output	Failure it prevents
Collaboration-pattern design checklist	Chapter 6	Before committing a pattern instance to production	The chosen pattern, agent specs, and budgets	A pass over nine items: pattern fit; termination conditions and cost budgets asserted by a test; failure handling; aggregation strategies and real diversity; observability; and justified complexity	A pattern that fails mid-production from an incomplete design

Artifact	Chapter	Use when	Inputs	Output	Failure it prevents
Memory-management design checklist	Chapter 7, with Figure 7.1 (the lifecycle) and <i>LLM-visible context versus runtime context</i>	Before deploying an agent with memory	The memory stores and their retention and access rules	A pass over 14 items: memory types, compaction, and lifecycle; retrieval and context budgets; access control, the context contract at agent boundaries, and write authority; forgetting, provenance, and privacy; and the memory metrics to monitor in production, the memory failure catalog, and regression cases	A store that leaks across namespaces, keeps what a deletion request should have removed, or cannot say where a fact came from

Artifact	Chapter	Use when	Inputs	Output	Failure it prevents
Orchestration design checklist	Chapter 8, with the per-run error budget under <i>Error budgets</i>	Before deploying an orchestrated workflow	The topology, flow shape, state, and retry policies	A pass over 20 items: a separate orchestration layer, topology fit, and explicit dependencies; retry and idempotency wiring with failure-injection tests; circuit breakers and fallbacks; durable state and the session handoff record; aggregation strategies and gated irreversible actions; and error budgets that drive degradation, with every decision traced and termination guaranteed	Production incidents from incomplete reliability wiring, or a workflow that fails by surprise instead of degrading on purpose

Artifact	Chapter	Use when	Inputs	Output	Failure it prevents
Cross-boundary interoperability design checklist	<i>Chapter 9</i>	Before a system starts calling agents it does not own	The agent contracts, registry, protocol adapters, and the trust boundary each call crosses	A pass over 14 items: agent contract completeness and registry health; least-privilege MCP scoping and Agent Card verification; adapter integrity and trace propagation; versioning and consumer-driven contract tests; transport security and task-level authorization; and input validation, audit logging, semantic alignment, and ownership	An integration that breaks silently when the peer is rewritten, or sends a token to an Agent Card it never verified

Artifact	Chapter	Use when	Inputs	Output	Failure it prevents
Evaluation specification	<i>Chapter 11, The five evaluation dimensions and Table 11.1</i>	Before measuring anything	The five evaluation dimensions and their priority order for the deployment context	A versioned document: metrics per dimension with targets, per-answer decision thresholds for metrics gated at run time, the per-case budget, and trials per case	A real improvement nobody can tell from moved goalposts
SHARP eval-suite properties	<i>Chapter 11, Properties of a good eval suite: SHARP and Designing your eval suite: the six steps, with Simulation-based evaluation</i>	Building an eval suite, or reviewing one before trusting its scores	The production task distribution, gold outputs or rubrics, reproducible fixtures, and trace instrumentation	A suite that is Sensitive, Holistic, Adaptive, Representative, and Probabilistic, built in six steps, with cases and trials sized from a pilot	A suite that flatters the system, or a gating decision made on noise

Artifact	Chapter	Use when	Inputs	Output	Failure it prevents
Judge calibration procedure	<i>Chapter 11, The calibration procedure, with the judge tiers in Table 11.6, Structured human evaluation for rubrics and inter-rater agreement, and Known judge biases</i>	Before a judge's score gates anything, and whenever the judge model, judge prompt, or rubric changes	The rubric, a powered held-out human-rated calibration set per dimension, and the judge	A per-dimension verdict against predeclared acceptance criteria (correlation, error, calibration, subgroup performance, and coverage), and the cheapest judge tier that clears it	Release gates resting on a judge nobody has checked against human ratings
Evaluation report and release card	<i>Chapter 11, Putting it all together: the evaluation report and release card; applied in Chapter 18, The last architecture review</i>	Reporting an evaluation run to engineers and to whoever decides the release	One record per spec metric per panel: estimate, interval, target, and direction	Each metric as met, missed, or inconclusive, and a card carrying the recommendation, population, baseline, the observation that reverses it, and the decision owner	An inconclusive result rounded up to a pass, or a stakeholder view hand-curved away from the record

Artifact	Chapter	Use when	Inputs	Output	Failure it prevents
Regression gate	Chapter 11, <i>The regression gate</i>	Before releasing any change to a prompt, model, agent, or orchestration	A predeclared primary metric, the smallest regression that matters, hard guardrails, baseline and candidate on paired cases, and a version record of each run	Regression, non-inferior, or inconclusive, feeding a ship, hold, narrow, or roll-back decision written down with what justified it	A cross-agent regression no component test sees, or an inconclusive interval read as a pass

Artifact	Chapter	Use when	Inputs	Output	Failure it prevents
One-engineer-week evaluation kit	<i>Chapter 11, Starting from zero: a one-engineer-week kit</i>	Bootstrapping evaluation from zero	A working agent and its production task categories	Level 2 of the MAS evaluation maturity model (automated correctness testing): correctness cases, a spot-check ritual, red-team scenarios, an evaluation specification, and a single-command runner	Shipping a system with no way to catch regressions

Artifact	Chapter	Use when	Inputs	Output	Failure it prevents
MAS evaluation design checklist	<i>Chapter 11, with Resource efficiency metrics, Robustness metrics (pass^k), Coordination metrics, and the role ablation under Testing whether coordination helped</i>	Before deploying a MAS to production	The eval suite and its instrumentation	A pass over 21 items: the evaluation specification and its five dimensions; sampling and trace capture; human and judge calibration; red-teaming and the degradation profile; coordination diagnostics, role ablation, and cost per successful outcome; and the regression gate, eval-set hygiene, the evaluation-improvement loop, and the measure-or-shape rule	Gating on a single run, or keeping a role that never paid for itself

Artifact	Chapter	Use when	Inputs	Output	Failure it prevents
Observability design checklist	Chapter 12, with <i>The four observation dimensions</i> for what to capture	Before calling a system production-ready	The tracing, cost-tracking, privacy, alerting, and runtime-control stack	A pass over 18 items: tracing; cost reconciliation and the infrastructure join; sampling and the unsampled authority record; privacy and evidence-plane health; dashboards, alerting, and reconstruction; and the execution-failure taxonomy, fix verification, and the runtime agent control patterns	Deploying with no way to diagnose a non-deterministic failure

Artifact	Chapter	Use when	Inputs	Output	Failure it prevents
Self-evolution design checklist	Chapter 13	Before declaring an adaptation loop genuinely self-evolving	The learnable surfaces and who or what may change each, and the regression suite	A pass over 16 items: the five-question self-evolution test; named surfaces and universal metadata; credit assignment and the experience-loop data-governance checklist; the six-step distillation pipeline with contraindications; the Evolution Contract, its gates, and rollback lineage; and forbidden moves and maturity fit	An implicit surface, a missing gate, or a rollback that restores the deployment but not the state the loop learned from

Artifact	Chapter	Use when	Inputs	Output	Failure it prevents
MAS security and privacy design checklist	Chapter 14	Before signing a system off as production-ready	Identity and capability-token machinery, isolation boundaries, AI bill of materials (AI-BOM), and cross-handoff data flows	A pass over 13 items: the tier on the three-tier threat staircase; identity and tool mediation; untrusted-content and planner-executor boundaries; message authentication and memory provenance; guardrail layering and control failure; and supply chain, the continuous red-team loop, privacy as contextual integrity, and compliance evidence	Shipping with standing credentials, untrusted tool output that can reach the planner, or a supply chain no one can revoke

Artifact	Chapter	Use when	Inputs	Output	Failure it prevents
Capacity plan	<i>Chapter 15, Capacity planning: from workload to budget</i>	Before the first production deploy, and again when fan-out, context size, or task volume changes materially	Eight inputs, from task volume to the latency service-level objective (SLO) and budget cap	Daily cost against the cap, peak tokens per minute against the provider quota, end-to-end p95 latency against the SLO, and the end-to-end completion rate	Budgeting on task count when token volume and fan-out drive the bill
Staging failure-injection cases	<i>Chapter 15, Breaking the runtime on purpose</i>	Before promoting a release	A staging deployment on the canary's pinned manifest, and eight injected failures, from a version change during suspension to a provider or regional outage	Each case asserting its invariant: no duplicated effect, no leaked lease, no reservation left open without cause, no run on a version it was not pinned to	Failures no unit test reaches, at the seams between a worker and its lease, a workflow and its pinned version, and a run and the client that started it

Artifact	Chapter	Use when	Inputs	Output	Failure it prevents
Deployment and scaling design checklist	Chapter 15, Table 15.8, with <i>Canary gates: behavioral and cost metrics, not just error rate</i> and the fleet-level error budget under <i>Committing to a service level you can keep</i>	Before calling a deployment production-ready	Which "required when" conditions hold	A pass over 21 items: secrets, deployable boundaries, and durable state; idempotent effects and budgets; release gating, the release manifest, and rollback; queues and broker delivery; scaling, rate limits, backpressure, and tenant isolation; and the canary gate, residency, and service levels	State lost on restart, or a rollback with no rule for in-flight work

Artifact	Chapter	Use when	Inputs	Output	Failure it prevents
Agent-facing surface requirements	<i>Chapter 17, What an agent-facing surface must state</i>	Before opening an API or product to agent callers	Each operation, and what integrators now learn only from documentation, support, and experience	A pass over eight statements, from preconditions and postconditions through reversibility classification, compensating actions, and receipts to structured errors that name the next valid actions	Agent traffic arriving before the contract it needs: an unsafe retry, or a partial effect nothing reverses
Engineering obligations for consequential systems	<i>Chapter 18, Engineering obligations</i>	Scoping a use case whose outputs or actions affect people, before the first prototype	The use case, the people it affects, and the effects it may commit	A pass over eight commitments, from auditability to retirement	An appeal path designed only after complaints, a handover nobody has tested, or a system that can only expand

Table B.3: The tools for verifying production readiness

Diagnosing and recovering from failure

Reach for these when something is wrong, or when you are instrumenting a system to catch it early. Table B.4 holds what to emit and measure, the taxonomies and mappings that pair what you can observe with its cause or its fix, and the controls and procedures that act on a failure.

Artifact	Chapter	Use when	Inputs	Output	Failure it prevents
Single-agent telemetry floor	Chapter 4, Table 4.5	Instrumenting a single agent, from the first prototype	The agent's run loop, its tool calls, its budget counters, and its escalation conditions	Six fields every run emits: <code>task_id</code> , <code>agent</code> , <code>tool_call</code> , <code>budget</code> , <code>termination_reason</code> , and <code>escalation</code>	Debugging a misbehaving agent by guesswork, from a run that cannot be reconstructed
Pattern failure signatures	Chapter 6, Table 6.5, with fixes under <i>Anti-patterns to avoid</i>	Instrumenting a collaboration pattern, or reading one that misbehaves	The pattern in use: Concurrent Specialists, Sequential Cascade, Task Decomposition, Debate and Reflection, Mixture of Experts, or Relay Handoff	Its characteristic failure and the signal that reveals it; four named anti-patterns (Parliament, Echo Chamber, Decomposition Tax, Infinite Refinement) carry the fix	A pattern failing in its known way while nobody watches the signal

Artifact	Chapter	Use when	Inputs	Output	Failure it prevents
Memory metrics to monitor in production	<i>Chapter 7, Memory metrics to monitor in production</i>	Instrumenting a deployed memory system	The memory system's design budgets	Retrieval precision, retrieval latency, store size and eviction rate, supersession lag, and consolidation yield, each alerted against a design budget	Silent memory degradation: index drift, stale facts, or forgetting that has quietly stopped
Memory failure catalog	<i>Chapter 7, Diagnosing memory failures</i> for the store and Table 7.2 for the context window	An agent answers plausibly from the wrong memory and nothing has thrown	The symptom, and the store's six instruments, from provenance to synthesized_ from back-pointers	One of eight failure modes (stale truth, over-retrieval, under-retrieval, context poisoning, namespace leakage, privacy retention, conflicting facts, confidence inflation), each with its signal and fix; in the window, poisoning, distraction, confusion, or clash, each with its defense	A memory fault that surfaces only as plausible answers built on the wrong foundation

Artifact	Chapter	Use when	Inputs	Output	Failure it prevents
Reliability-pattern catalog	Chapter 8, Table 8.3, <i>When the call itself is wrong</i> , the plan validation under <i>Centralized orchestration</i> , and Table 8.4 for durable-execution engines	Choosing a recovery pattern for an orchestration failure	One of Table 8.3's eight failure modes	The pattern that closes it: idempotency key, retry with backoff and jitter, fallback chain, hedging, verification gate, circuit breaker, graceful degradation, or durable orchestrator state; a call the model got wrong is repaired, not retried; and a planner's output is validated before any task runs	A long pipeline less reliable than its individual components suggest
Four-phase failure taxonomy	Chapter 11, <i>The four phases of agentic failure</i> and Table 11.4	From the evaluation side: deciding what an eval suite must catch, or naming the phase where a failed run diverged	A failing trace, or the suite's case inventory	The phase where the divergence sits (interpretation, reasoning, action, or perception) and one of 20 named failure modes, five per phase	An eval suite silent on a whole class of failure

Artifact	Chapter	Use when	Inputs	Output	Failure it prevents
Evaluation-improvement loop	Chapter 11, Figure 11.2 and Table 11.8	A production slice scores below the evaluation specification's target	Online verdicts and user feedback per stratum, and the failing traces with their recorded tool responses	One pass of six steps, from selecting a slice to confirming online	Changes shipped together that cannot be told apart, or a gate that passes while the live slice does not move
Agent token-spend taxonomy	Chapter 12, Table 12.3	Explaining a workflow's cost, or choosing which optimization to make	Priced model-call spans, plus the harness's own counts of scaffolding, carried context, and fresh input	Spend split by purpose (task work, coordination, oversight, context upkeep) and by composition (scaffolding, carried context, fresh input, reasoning, visible output), with a rework flag and the lever each share points to	Double-counting by mixing the two axes, or booking the cost of judging the system as the cost of running it

Artifact	Chapter	Use when	Inputs	Output	Failure it prevents
Execution-failure taxonomy	<i>Chapter 12, Classifying the failure; counterfactual probes under Credit assignment across independently changing boundaries in Chapter 13</i>	From telemetry, during an incident: triaging a failure in a running workflow	An observed failure and its telemetry: spans, inter-agent messages, cost metrics, timing across dependencies	One of six categories (agent-level, coordination, feedback loop, resource exhaustion, cascading, or silent) with where to look and the usual fix, then an attribution to an agent and step, held as a hypothesis to verify	Investigating from the wrong evidence, or missing a silent failure
Signal-to-decision mapping	<i>Chapter 12, From signal to decision and Table 12.5</i>	Choosing between retry, rollback, throttle, and escalation when an alert fires	The firing signal, one of nine from an error span to a projected telemetry budget breach	The decisions each signal supports, and the owner each family of signals routes to	Retrying under incident pressure when the signal calls for a rollback, a throttle, or a human

Artifact	Chapter	Use when	Inputs	Output	Failure it prevents
Runtime agent control patterns	<i>Chapter 12, Runtime agent control and guardrails</i>	Enforcing the evaluation specification's thresholds on live traffic	The specification's per-answer decision thresholds and per-case budget and time limits	Three patterns, cheapest first (output validation, policy-gate agents, circuit-breaker coupling), each with a placement (input, tool-call arguments, handoff, or output) and an action when it fires	Detectors where the team wanted controls, or a safety failure regenerated until it clears the check
Technique -to- surface matrix	<i>Chapter 13, the decision tree under Matching the fix to the cause and Table 13.3</i>	Picking the technique that fixes a diagnosed quality problem	The primary symptom, one of five from wrong on uncovered edge cases to wrong tone or style	The shallowest technique that can plausibly close the gap, with the surface it edits, the signal it consumes, its cost, and its main governance concern	Reaching for fine-tuning to compensate for a prompt, retrieval, or contract defect

Artifact	Chapter	Use when	Inputs	Output	Failure it prevents
Compromised-agent containment sequence	Chapter 14, Containing a compromised agent	An agent is suspected compromised in production	The agent's capability grants and non-human identity credentials with their delegation lineage, the audit log and traces, write-time memory provenance, and a clean snapshot	Seven steps in order: stop new effects, preserve the record, revoke down the delegation lineage, isolate derived state, rotate what it could reach, restore from a clean point, and promote the incident into the red-team curriculum	Pulling controls in an order that overwrites the evidence, or leaving a delegated sub-agent's grants live

Table B.4: The tools for diagnosing and recovering from failure

Securing and governing change

A live system has to withstand attack and change safely. Table B.5 starts with learning: what the system may keep and learn from, and what it may change about itself. It then places the threats by tier, names the controls that answer them and the loop that tests those controls, and says who owns each operational risk. Its last rows cover what a release contains and how it is backed out. They close on who may act on whose behalf across organizations, how human feedback becomes evidence, and which components to retest when a model, protocol, or policy moves.

Artifact	Chapter	Use when	Inputs	Output	Failure it prevents
Dream pass	<i>Chapter 7, Memory consolidation: the dream pass</i>	A long-lived memory store is accumulating contradictions and undifferentiated episodes	The memories added since the last pass, and the store they join	An offline pass of four operations (reconcile, synthesize, reflect, demote and prune), with demotion reversible through synthesized_from back-pointers, at a cadence tuned on new memories per pass and the fraction that change something	Retrieval that surfaces a stale fact beside the current one, with nothing to say which to trust

Artifact	Chapter	Use when	Inputs	Output	Failure it prevents
Measure-or-shape rule	<i>Chapter 11, Measure or shape, never both; restated in Chapter 13, Goodhart's law in MAS</i>	Harvesting production traces, as eval cases or as data the system learns from	Each trace's admission record and its deduplication key	Every trace assigned at admission to one side, measuring or shaping, split by cluster, with dataset versions in the version record and the shaping side under the experience-loop data-governance checklist	A suite that scores well on a slice because the system has already seen its answers
Evidence-minimization matrix	<i>Chapter 12, Table 12.4</i>	Designing the telemetry pipeline, when deciding what each record may carry	The five evidence classes the system emits, from structural span attributes to high-impact effect audits	A capture default per class, from off for sensitive payloads to on and append-only for effect audits, with its purpose and control	A trace store that leaks personal content or keeps it past its purpose

Artifact	Chapter	Use when	Inputs	Output	Failure it prevents
Experience-loop data-governance checklist	<i>Chapter 13, Data governance for the experience loop</i>	Before any stream of traces, labels, or feedback feeds a learning loop	The experience stream and its rules	A pass over six items: capture; filtering and deduplication; privacy and consent; label provenance; poisoning resistance; and retention and reversibility	Learning from sanitized, biased, or poisoned experience, or being unable to remove what a withdrawn dataset taught
Six-step distillation pipeline	<i>Chapter 13, Figure 13.3</i>	Turning a human-agent working session into reusable knowledge	The trace of joint work, historical traces, and a held-out eval set	Through capture, segment, extract, generalize, validate, and encode, a knowledge candidate (the lesson with its applicability, evidence, contraindications, and governance fields), reviewed before anything is installed on a surface	A raw transcript poured into memory or a fine-tuning corpus, or one incident over-generalized

Artifact	Chapter	Use when	Inputs	Output	Failure it prevents
Evolution Contract	<i>Chapter 13, The seven clauses,</i> reviewed through the chapter's five-question test; Table 13.4 (worked example) and Table 13.6 (the minimum-viable form)	Authorizing change in a self-evolving system	The seven learnable surfaces (Table 13.2) and the eval-suite companion surface	A versioned agreement of what may change on each surface, on what evidence, behind which eval gates and deployment gates, who approves, how it rolls back, and what is forbidden; one page at self-evolution L2 (assisted authoring), enforced in continuous integration (CI) from self-evolution L3 (system-proposed)	Ungoverned self-modification that drifts past spec or games its own gates

Artifact	Chapter	Use when	Inputs	Output	Failure it prevents
Three-tier threat staircase	Chapter 14, Figure 14.1 and Table 14.1, with the lethal trifecta under Tier 2	Placing a system, or a newly published attack, on the threat surface before choosing controls	The capability the system has acquired	A tier (1 generation, 2 effects, 3 collaboration), the threats new at it and inherited from below, and the mitigations that match	Counting agents instead of capability, or hardening each component alone while a Tier 3 pattern runs through a handoff no component sees

Artifact	Chapter	Use when	Inputs	Output	Failure it prevents
AM/RG control catalog	Chapter 14, Table 14.2 and Figures 14.2–14.3	Choosing architectural mitigations (AM) and runtime guardrails (RG) and mapping them to threats	The threat model and the system architecture	Seven of each, from AM1 capability tokens to AM7 sandboxed untrusted regions and from RG1 tool-call mediator to RG7 programmable rails, and which of them answer each category of the OWASP Top 10 for Agentic Applications; cascading failures and rogue agents are only partly covered, and emergent collusion stays open	Architectural weakness that cannot be retrofitted, or runtime gaps the architecture missed

Artifact	Chapter	Use when	Inputs	Output	Failure it prevents
Minimum viable MAS security baseline	<i>Chapter 14, Table 14.3 and Trust boundaries and authority boundaries</i>	Establishing a defensible security floor before any other hardening	The system's user input and model output, tools, inter-agent bus, shared memory, irreversible actions, and untrusted-content paths	The seven controls that close the highest-impact attack classes first	Hardenin g further while one of the seven is missing, a structural gap no runtime filtering alone can close
Continuous red-team loop	<i>Chapter 14, Figure 14.4, Table 14.4, and Red-team finding triage</i>	Standing up adversarial testing, and again whenever a new public incident, tool, or model lands	The threat model and the public incident canon, tier-mapped benchmarks, and the running system with its tools and memory	Findings routed to an architectural, runtime, or policy fix, each leaving a regression test in the suite	A pre-launch audit mistaken for a posture, or findings all treated alike until the program is noise

Artifact	Chapter	Use when	Inputs	Output	Failure it prevents
Production risk register	Chapter 15, Table 15.5, with its ownership axes from <i>Three levels of cost control</i>	Staffing a platform team and assigning operational risk	Nine production risk classes, from cost runaway to broker durability mismatch, and three ownership axes: agent engineer, workflow owner, and platform/site reliability engineering (SRE), with security and legal as co-owners	Risk classes mapped to owners and control surfaces, with a weekly, quarterly, and per-release cadence	Operational risks that no one owns until they fire

Artifact	Chapter	Use when	Inputs	Output	Failure it prevents
Release manifest	<i>Chapter 15, Table 15.6, with the rollback responses in Table 15.7</i>	Defining what a release is, from the first deploy, and backing one out while workflows are running	Ten release surfaces, from code and orchestration to evaluation set	One fingerprint on every span; per surface, whether it rolls back alone and what it couples to; per state class (in-flight, cached, queued, and memory-and-index state), a default rollback response	A behavior change with no version to blame, or a rollback that restores the binary and not the state beside it
Authority-plane controls	<i>Chapter 17, From principal to effect and Figure 17.4</i>	Designing delegated action that crosses organizations, before the controls are collapsed into "auth"	The identities in a run, from principal to sub-agent or tool, and the effect to be committed	Eight controls kept distinct: authentication, authorization, delegation, consent and intent, effect binding, contextual policy, credential lifecycle, and evidence and recourse	A valid credential standing in for the principal's intent, or a stale approval executing against changed terms

Artifact	Chapter	Use when	Inputs	Output	Failure it prevents
Feedback transaction	<i>Chapter 17, The interface is also a learning boundary; Chapter 13, Human feedback taxonomy, for the signal's meanings</i>	Capturing a correction, approval, or preference that might be reused beyond the current run	The human signal, at the moment it is given	A record of seven fields, from target to the later observation that would confirm it helped	An approval, silence, or undo read as an unqualified reward, or a one-run correction becoming policy with no way to trace and revoke it

Artifact	Chapter	Use when	Inputs	Output	Failure it prevents
Assumption register	Chapter 17, Table 17.3, and <i>Volatile implementation, durable obligations</i> for what must survive	Deciding whether a component is still needed after a new model, protocol, or policy ships	Each component's assumption, its class (capability workarounded, product obligation, ecosystem hedge, or more than one), the evidence today, and the cost if wrong	A retest trigger, the experiment that would change the decision, and an owner and disposition	Keeping a workaround and the model has outgrown, or retiring a product obligation because a benchmark improved

Table B.5: The tools for securing and governing change

Planning the maturity roadmap

The spectrum, ladders, maturity models, and adoption order in Table B.6 each name a sequence of levels or steps, so you can place your system today and see the next one. On most of them the right position is the lowest that does the job, and each step up has a trigger and a cost to weigh before you commit to it. Three of them number their levels 1 to 5 and measure different things: what a system owns, how rigorously it is evaluated, and how much authority its learning loop holds.

Artifact	Chapter	Use when	Inputs	Output	Failure it prevents
Agenticity spectrum	Chapter 1, Table 1.1 and Figure 1.1	Placing a system, and deciding whether the problem warrants moving up a level	What the system would take responsibility for, and the task's depth and decomposability	An agenticity level (L1 assisted, L2 goal-directed, L3 coordinated, L4 adaptive, or L5 fully autonomous), starting from the minimum viable one, and whether it matches the task	Too much agenticity, where coordination overhead exceeds the value added, or too little, a brittle design whose errors are silent

Artifact	Chapter	Use when	Inputs	Output	Failure it prevents
Interoperability ladder	Chapter 9, Table 9.1	Scoping how far interoperability must reach	The integration's boundary and the guarantees it needs across it	The rungs it must reach (wire, contract, semantic, trust, operations), the mechanism that covers each, and where coverage stays partial	An integration that looks interoperable until the two sides read a field differently, or ownership agreements found missing mid-incident

Artifact	Chapter	Use when	Inputs	Output	Failure it prevents
Interoperability adoption order	Chapter 9, <i>Minimum viable adoption: what to do first</i>	Starting from one agent calling tools and a few internal services	The most-used agent, which calls cross a team boundary or outlast a load balancer's timeout, and whether membership , ownership, or routing changes at runtime	Five steps in onboarding order, each with a time budget: MCP servers for external tools, one agent contract, A2A only at a boundary that matters, a registry from three dynamically discovered agents, and an adapter from day one	Starting from the architecture diagram and instantiating every box

Artifact	Chapter	Use when	Inputs	Output	Failure it prevents
MAS evaluation maturity model	Chapter 11, <i>The maturity model for MAS evaluation</i>	Planning evaluation investment for the product's stage	Product stage, team size, risk surface	A current evaluation-maturity level, from Level 1 (manual spot-checking) to Level 5 (closed-loop improvement), and the next level's investment	Building Level 4's continuous evaluation pipeline before the product stops pivoting, or under-evaluating a system already in production

Artifact	Chapter	Use when	Inputs	Output	Failure it prevents
Self-evolution maturity model	Chapter 13, Figure 13.5 and Table 13.5	Deciding how far to automate learning	Specification stability, eval-suite maturity, team staffing, and whether stop-and-restore is drilled	A self-evolution level, from L1 (human-authored) to L5 (broadly auto-deployed), who holds change authority on each surface at that level, and the trigger, team, and risk of the next step	Automating learning before a stable specification, an offline eval suite, or an Evolution Contract exists, or granting the loop more authority than its monitoring and drilled stop-and-restore can back

Artifact	Chapter	Use when	Inputs	Output	Failure it prevents
Capability-to-contribution ladder	Chapter 18, Summary, with <i>Next steps for readers</i> for the one next action	Placing a system and choosing what to add next	What the system makes explicit, who operates and answers for it, and whom it affects	A stage (capability, system, product, or contribution) and what the next stage adds; disclosure, affected people, and recourse constrain the use case before the first prototype	Mistaking a running capability for a system, or a deployed, documented system for a contribution

Table B.6: The tools for planning the maturity roadmap

Each row points to where the book builds its tool and works it through, usually a single chapter. Reach for a group when you know the job in front of you; reach for the chapter when you need to understand why the tool has the shape it does.

